

FleetIQ

AI-Powered Fleet Operations and Operational Intelligence Platform

Joseph Karianjahi Njeri

June 22, 2026

Contents

1	Executive Summary	3
2	Business Problem	3
3	Technology Stack	3
3.1	Backend	3
3.2	Database	4
3.3	Analytics and Machine Learning	4
3.4	Frontend	4
3.5	Deployment	4
4	System Architecture	4
5	Core Domain Models	5
5.1	Vessel	5
5.2	Voyage	5
5.3	TelemetryRecord	6
5.4	OperationalAlert	6
6	Fleet Health Analytics	6
7	Operational Alert Engine	7
7.1	Engine Overheat	7
7.2	Weather Warning	7
7.3	Fuel Anomaly	7
7.4	Delay Risk	7
7.5	Speed Anomaly	7
8	Synthetic Data Generation	7
9	REST API Architecture	8
10	Dashboard Platform	8
11	Machine Learning Implementation	9
12	Testing Strategy	9

13 Deployment Architecture **10**

14 Future Roadmap **10**

 14.1 Machine Learning 10

 14.2 Artificial Intelligence 10

 14.3 Platform Expansion 11

15 Current Development Status **11**

1 Executive Summary

FleetIQ is a cloud-deployed operational intelligence platform designed to monitor fleets, analyse telemetry data, detect operational risks, and support data-driven decision-making through analytics, machine learning, and interactive dashboards.

The platform demonstrates how modern operational systems can transform raw telemetry into actionable intelligence using:

- Fleet analytics
- Operational alerting
- Health monitoring
- Machine learning
- REST APIs
- Interactive dashboards
- Cloud-native deployment

FleetIQ is currently deployed on Railway using PostgreSQL and production-grade Django infrastructure.

2 Business Problem

Modern fleets generate large volumes of operational data, including:

- Location data
- Speed measurements
- Fuel consumption
- Equipment temperatures
- Operational events
- Environmental conditions

While this data is valuable, operators often struggle to transform it into actionable insights.

FleetIQ addresses this challenge by:

- Monitoring fleet health
- Detecting operational risks
- Providing centralized analytics
- Supporting predictive intelligence
- Delivering operational dashboards

3 Technology Stack

3.1 Backend

- Python

- Django
- Django REST Framework

3.2 Database

- PostgreSQL

3.3 Analytics and Machine Learning

- pandas
- scikit-learn
- joblib

3.4 Frontend

- HTML
- CSS
- JavaScript
- Chart.js

3.5 Deployment

- Railway
- Gunicorn
- WhiteNoise

4 System Architecture

```

Vessel
  |
Voyage
  |
TelemetryRecord
  |
Operational Alert Engine
  |
OperationalAlert
  |
Analytics APIs
  |
Fleet Dashboard
  |
Machine Learning

```

The architecture separates:

- Operational data
- Business logic

- Alert generation
- Analytics services
- Machine learning services
- Presentation layer

5 Core Domain Models

5.1 Vessel

Represents a physical ship within the fleet.

Responsibilities:

- Vessel metadata
- Operational status
- Capacity management
- Fleet ownership

Example fields:

- Name
- IMO number
- Vessel type
- Capacity
- Fuel capacity
- Status

5.2 Voyage

Represents an operational journey.

Responsibilities:

- Port tracking
- Voyage status
- Travel history
- Operational reporting

Example fields:

- Departure port
- Destination port
- Departure time
- Estimated arrival
- Actual arrival

- Distance
- Status

5.3 TelemetryRecord

Stores operational telemetry.

Responsibilities:

- Operational monitoring
- Analytics support
- Alert generation
- Machine learning features

Example fields:

- Timestamp
- Latitude
- Longitude
- Speed
- Fuel consumption
- Engine temperature
- Weather risk score

5.4 OperationalAlert

Stores operational warnings.

Current implemented alert types:

- Engine Overheat
- Fuel Anomaly
- Weather Warning
- Delay Risk
- Speed Anomaly

Planned future alert types:

- Route Deviation
- Predictive Maintenance Alerts
- Dynamic Risk Alerts

6 Fleet Health Analytics

FleetIQ includes fleet-wide operational health monitoring.

Current capabilities:

- Fleet health distribution
- Vessel health scoring
- Alert-driven risk monitoring
- High-risk vessel identification
- Operational trend monitoring

7 Operational Alert Engine

Location:

```
operations/services/alert_engine.py
```

Purpose:

Automatically analyse telemetry data and generate operational alerts.

Current implemented rules:

7.1 Engine Overheat

```
if engine_temperature >= 95:
    create_engine_overheat_alert()
```

7.2 Weather Warning

Generated when weather risk exceeds defined thresholds.

7.3 Fuel Anomaly

Generated when fuel consumption exceeds expected operating ranges.

7.4 Delay Risk

Generated when voyage timing indicates elevated delay probability.

7.5 Speed Anomaly

Generated when vessel speed exceeds configured operational thresholds.

8 Synthetic Data Generation

FleetIQ includes a configurable synthetic data engine.

Current capabilities:

- Vessel generation
- Voyage generation
- Telemetry generation
- Operational alert generation
- Fleet health simulation

Current dataset generation supports:

- 50+ vessels
- Multiple voyages per vessel
- Telemetry time series
- Operational alert creation

9 REST API Architecture

FleetIQ follows an API-first design.

Dashboard APIs:

```
/api/dashboard/kpis/  
/api/alerts/latest/  
/api/alerts/summary-by-type/  
/api/alerts/summary-by-severity/  
/api/voyages/status-summary/  
/api/alerts/over-time/  
/api/alerts/top-vessels/
```

Vessel APIs:

```
/api/vessels/  
/api/vessels/<id>/  
/api/vessels/<id>/voyages/  
/api/vessels/<id>/alerts/  
/api/vessels/<id>/kpis/  
/api/vessels/<id>/health-status/
```

Fleet APIs:

```
/api/fleet/health-distribution/
```

10 Dashboard Platform

FleetIQ includes a production dashboard.

Current features:

- KPI cards
- Fleet health analytics
- Latest alerts
- Alert summaries
- Alerts over time
- Top vessels by alerts
- Vessel detail pages
- Voyage history views
- Interactive drill-down navigation

11 Machine Learning Implementation

FleetIQ contains a working machine learning pipeline.

Generated features:

- Average speed
- Average engine temperature
- Average weather risk
- Average fuel consumption
- Total alerts

Current target:

```
High Risk Vessel
```

Current algorithm:

```
Random Forest Classifier
```

Libraries:

- scikit-learn
- pandas
- joblib

Model persistence:

```
operations/ml_model/vessel_risk.pkl
```

Current capabilities:

- Dataset generation
- Feature engineering
- Model training
- Model serialization

12 Testing Strategy

Current test coverage includes:

- Models
- Serializers
- APIs
- Alert engine
- Business logic

Tools:

- Django TestCase

- coverage.py

Coverage:

96% Coverage

13 Deployment Architecture

FleetIQ is deployed using Railway.

Production stack:

- Railway
- PostgreSQL
- Gunicorn
- WhiteNoise
- Environment variables
- Production Django settings

Deployment pipeline:

<pre>GitHub Railway PostgreSQL Production Application</pre>

14 Future Roadmap

14.1 Machine Learning

- Predictive maintenance
- ETA prediction
- Fuel consumption forecasting
- Operational risk forecasting
- Vessel health prediction

14.2 Artificial Intelligence

- AI-generated operational summaries
- Natural language operational queries
- Intelligent anomaly explanations
- Operational recommendations

14.3 Platform Expansion

- Real-time telemetry streaming
- Docker deployment
- Role-based access control
- Authentication and authorization
- Multi-tenant architecture

15 Current Development Status

FleetIQ has evolved from a prototype into a deployed operational intelligence platform.

Completed:

- Cloud deployment
- PostgreSQL integration
- Fleet analytics
- Vessel analytics
- Fleet health scoring
- Alert engine
- REST APIs
- Dashboard platform
- Machine learning pipeline
- Synthetic data generation

Current operational pipeline:

```
Telemetry
  |
Alert Engine
  |
Operational Alerts
  |
Analytics APIs
  |
Dashboard
  |
Machine Learning
```

FleetIQ now serves as a realistic demonstration of modern operational analytics, software engineering, cloud deployment, and machine learning integration within a single enterprise-style platform.